

資料庫系統期末專題 Report 4

錢錢錢市－股票模擬系統

第八組

41347001S 楊凱臣 41347009S 周聖詠
41347011S 許育綸 41347041S 盧人豪

1 附上前幾次報告，從系統簡介、ER-diagram 設計、到轉換成 SQL relational database create table 的設計。

由於報告頁數很多，因此將前幾次的報告部分節錄放在附錄中：

- Report 1：系統簡介
- Report 2：ER-diagram 設計
- Report 3：SQL relational database create table 的設計

2 系統安裝說明

2.1 專案程式碼連結

專案程式碼已公開於 GitHub，包含前端與後端的完整原始碼：<https://github.com/Eason777777/stock-sandbox>。

2.2 系統需求

整體需求概述

項目	需求
Python	3.10 以上
Node.js	v20.19.0 或 v22.12.0 以上
套件管理	pnpm（前端）、pip（後端）
網路	需加入 Tailscale 虛擬私有網路，才能連線至資料庫
資料庫	MySQL（透過 HTTP SQL API 存取，地址 <code>sql-api.shiragaserver.lan</code> ）

後端相依套件一覽

套件	用途
fastapi	Web API 框架
uvicorn[standard]	ASGI 伺服器
httpx	非同步 HTTP 客戶端 (連接 SQL API)
python-dotenv	載入 .env 環境變數
passlib / bcrypt==4.0.1	密碼雜湊

前端主要相依套件

套件	用途
vue@3	UI 框架
vue-router@5	前端路由
klinecharts@9.8.5	K 線圖繪製 (Canvas)
tailwindcss@4	CSS Utility 框架
vite@8	開發建置工具
vitest	單元測試框架
oxlint + eslint	程式碼 Lint 工具
prettier	程式碼格式化

2.3 後端安裝與啟動 (FastAPI)

後端為 FastAPI 應用程式，以 uvicorn 作為 ASGI 伺服器。

```

1 # 1. 複製專案
2 git clone https://github.com/Eason777777/taiwan-stock-sandbox
3 cd taiwan-stock-sandbox
4
5 # 2. (建議) 建立虛擬環境
6 python -m venv .venv
7 source .venv/bin/activate # Windows: .venv\Scripts\activate
8
9 # 3. 安裝相依套件
10 pip install -r requirements.txt
11
12 # 4. 設定環境變數 (預設已有 .env)
13 # 若需覆寫 SQL API 位址:
14 export SQL_API_URL=http://sql-api.shiragaserver.lan
15
16 # 5. 啟動開發伺服器
17 uvicorn app.main:app --reload

```

後端預設監聽 <http://localhost:8000>，互動式 Swagger 文件位於 <http://localhost:8000/docs>。

2.4 前端安裝與啟動 (Vue 3 + Vite)

```
1 cd frontend
2
3 # 安裝相依套件 (需 pnpm)
4 pnpm install
5
6 # 啟動開發伺服器 (http://localhost:5173)
7 pnpm dev
8
9 # 建置正式版
10 pnpm build
```

2.5 驗證連線

```
1 # 測試 Tailscale + SQL API 連線
2 curl -X POST http://sql-api.shiragaserver.lan/query \
3   -H "Content-Type: application/json" \
4   -d '{"sql": "SHOW DATABASES"}'
```

2.6 執行測試

```
1 # 後端單元測試 (不需 DB)
2 pytest tests/unit/
3
4 # 後端整合測試 (需 Tailscale + 資料庫)
5 pytest tests/integration/
6
7 # 前端單元測試
8 cd frontend
9 pnpm test
10
11 # 後端 Smoke Test (需先啟動 uvicorn)
12 python backend_tests/smoke_test.py
```

3 系統功能評估（完成度，可擴展改進部份）

3.1 完成度評估

3.2 可擴展改進部分

4 介面使用說明

5 實際工作分配及工作份量比重

6 個人心得

6.1 楊凱臣

6.2 周聖詠

6.3 許育綸

6.4 盧人豪

A 附錄：Report 1

資料庫系統期末專題 Report 1

錢錢錢市—股票模擬系統

第八組

41347001S 楊凱臣 41347009S 周聖詠
41347011S 許育綸 41347041S 盧人豪

1 專題動機與目標

在這個日新月異的時代，愈來愈多人想學習股票交易，但又不知道該如何下手。雖然市面上已有部分模擬交易平台，但多數僅提供即時或近期資料，缺乏讓使用者回到特定歷史時間點、以真實市場情境反覆驗證判斷的環境。

因此本專題希望設計一套以台灣股市真實歷史資料為基礎的股票模擬系統，讓使用者選擇過去的時間點，透過實際下單與觀察 K 線、均線等技術指標，在零風險的環境下反覆驗證自己對市場走勢的判斷，建立對股票交易的直覺與理解。

本系統的目標如下：

1. 提供以真實歷史資料為基礎的虛擬交易環境。
2. 讓使用者體驗完整的交易流程，包含下單、撮合與損益計算。
3. 透過反覆模擬，協助使用者建立對 K 線與市場走勢的判斷能力。

2 系統使用者

1. **一般使用者**：對股票交易有興趣的新手或初學者。可註冊帳號、建立模擬存檔、進行模擬、進行虛擬股票交易，並查詢持股與損益紀錄。

3 系統主要功能

1. **帳戶管理**：使用者可註冊與登入帳號，系統記錄使用者身份資訊作為交易的基礎。
2. **模擬存檔管理**：使用者可建立多個獨立的模擬存檔，每個存檔擁有各自的時間軸、資金與持股狀態，互不影響。

3. **時間軸控制**：系統會隨機選擇一個過去的歷史日期作為模擬起點，使用者能手動推進模擬時間，每次推進一個交易日，依序經歷盤前、開盤、盤中、收盤、盤後五個階段。
4. **資金管理**：每個存檔包含存款戶與交割戶，使用者可在兩者之間進行轉帳，股票交易僅從交割戶扣款。
5. **股票查詢**：使用者可透過搜尋或類股分類瀏覽股票，並查看歷史 K 線圖與均線等技術指標。
6. **自選股管理**：使用者可將感興趣的股票加入自選清單，方便快速追蹤。
7. **下單與撮合**：使用者可提交限價單或市價單進行買賣，系統依照當日開盤價或收盤 OHLC 進行撮合。未成交的委託可查詢或撤銷。
8. **持股與損益查詢**：使用者可查看目前持有的股票明細，包含庫存量、平均成本與未實現損益，以及歷史交易紀錄與已實現損益。
9. **模擬回顧**：模擬結束後，使用者可查看整段模擬期間的資產走勢、交易紀錄與損益總結，了解自己的操作成效。

4 需求功能分析

4.1 功能性需求

1. 使用者可註冊與登入帳號。
2. 使用者可建立與管理多個模擬存檔。
3. 系統隨機選擇歷史時間點開始模擬。
4. 使用者可手動推進模擬交易日。
5. 使用者可在存款戶與交割戶之間進行轉帳。
6. 使用者可搜尋股票、瀏覽類股分類與查看 K 線圖。
7. 使用者可管理自選股清單。
8. 使用者可提交限價單或市價單，並查詢與撤銷待成交委託。
9. 系統依照當日真實歷史價格自動撮合委託單。
10. 使用者可查詢持股明細與損益紀錄。
11. 使用者可在模擬結束後查看整體模擬回顧。

4.2 非功能性需求

1. **Security**：使用者名稱須唯一，系統須拒絕重複註冊。
2. **Security**：密碼須符合最低強度要求。
3. **Security**：使用者只能存取自己的模擬存檔與資料。
4. **Reliability**：交易撮合須以 Transaction 處理，確保資料一致性。
5. **Reliability**：交割戶餘額不足時系統須拒絕成交，餘額不得為負數。
6. **Reliability**：同一委託單不得被撮合兩次。
7. **Maintainability**：各模擬存檔之間的資料須完全獨立，互不影響。

5 資料分析

本系統初步分析後，主要資料對象如下：

使用者相關資料

1. **使用者**：記錄帳號與密碼，作為登入驗證的基礎。

股票相關固定資料

1. **股票**：記錄台股上市股票的基本資料，如代號、名稱、所屬類股。
2. **歷史價格**：記錄每支股票在每個交易日的開盤價、最高價、最低價、收盤價、成交量等資訊。

模擬存檔相關資料

1. **模擬存檔**：記錄一組模擬的狀態¹、存檔 ID、玩家 ID。每位使用者可擁有多個獨立的存檔。
2. **帳戶**：每個存檔擁有存款戶與交割戶²，分別記錄各自的餘額。
3. **委託單**：記錄使用者在該次模擬存檔中某個模擬日提交的買賣委託³。
4. **交易紀錄**：交易記錄：記錄使用者在該次模擬存檔中的委託單 ID，以及包含成交金額、手續費、證交稅等相關費用。此資料描述委託單成交這件事本身。

¹狀態，指模擬日期、起始設定與遊玩狀態（進行中/已結束）等資訊。

²存款戶係指個人資金帳戶，交割戶係指存放交易股票所支出、領取資金的帳戶，使用者能互相進行轉帳操作。

³委託單具體而言會有：下單時間（盤前中後）、股票 ID、下單類型（限價、市價）、下單金額、股票數量（張）、下單狀態（買或賣）、委託狀態（待成交/已成交/已撤銷/已失效）等資訊。

5. **持股**：記錄使用者在該次模擬存檔目前持有某支股票的數量與平均成本。
6. **自選股**：記錄使用者在該次模擬存檔中所關注的股票清單。此資料描述存檔與股票之間的關注關聯。

6 資料關聯分析

1. 一位使用者可擁有多個模擬存檔。
2. 一個模擬存檔可擁有多個委託單、持股、自選股。
3. 一個模擬存檔擁有一個帳戶（包括存款戶與交割戶），一個帳戶擁有一個模擬存檔。
4. 一個模擬存檔可關注多支股票，一支股票也可被多個模擬存檔關注，兩者之間存在多對多關係，透過自選股來表示。
5. 一筆委託單可有至多一個交易記錄。
6. 一個股票對應多筆歷史價格，每個交易日一筆。

7 運作規則

1. 使用者須手動推進模擬時間，系統不會自動前進，依序經歷盤前、開盤、盤中、收盤、盤後五個階段，盤後之後進入下一天並回到盤前狀態。
2. 終止條件：直到到達資料集最新日期或者餘額不足。
3. 委託單僅支援當日有效 (ROD)，收盤後未成交的委託單自動失效。
4. 股票買賣最小單位為 1,000 股（一張），不接受零股。
5. 買進委託僅從交割戶扣款，若交割戶餘額不足則無法成交 (T+0 機制)。
6. 當使用者要進行買賣時有兩種委託單可選：
 - (a) 限價單
 - 限價買單：買入價格須高於或等於當日最低價才可成交。
 - 限價賣單：賣出價格須低於或等於當日最高價才可成交。
 - (b) 市價單：隨機以當日最高價與最低價之間價格成交。
7. 使用者提交委託單時，委託價格須在當日漲跌停範圍內。系統依據前一交易日的收盤資料取得當日漲跌停價格，超出範圍的委託單將被拒絕。

8. 手續費為買賣各 0.1425%（未滿 20 元以 20 元計），證交稅於賣出時收取（股票 0.3%）。
9. 交易紀錄需經委託單成交後才會產生。
10. 每個模擬存檔的時間軸與資產狀態互相獨立，互不影響。

B 附錄：Report 2

資料庫系統期末專題 Report 2

錢錢錢市—股票模擬系統

第八組

41347001S 楊凱臣 41347009S 周聖詠
41347011S 許育綸 41347041S 盧人豪

1 系統中的資料需求分析

1.1 使用者相關資料

1. 使用者憑證 (User)：負責記錄玩家登入系統所需之核心帳號與安全驗證資訊，並負責控管系統的登入安全狀態，包含：
 - `user_id`：使用者唯一識別碼 (Primary Key)。作為系統底層資料表關聯與後端撮合邏輯運算時的防呆主鍵，此欄位不會對使用者暴露。
 - `account`：登入帳號。具備唯一性 (Unique Constraint)，為玩家登入時的識別依據，並作為系統的預設顯示名稱。
 - `password`：安全雜湊密碼。經雜湊處理後儲存，確保無明碼外洩之資安風險。
 - `session_id`：登入憑證流水號 (Token)。作為前端與後端 API 溝通時的身分授權憑證。使用者每次成功登入皆會生成新憑證並「覆蓋」此欄位，藉此實作「單一登入強制控管」，嚴格防止同一帳號於多台裝置同時交易可能引發的 Race Condition。

1.2 股票相關固定資料

1. 股票基本面 (Stock)：儲存台股各檔股票之靜態與基礎資訊，做為系統唯讀的字典，包含：

- `stock_id`：股票代碼 (Primary Key)。使用 VARCHAR 型別以相容台股特殊代碼格式。如 2330。
 - `stock_name_zh`：中文簡稱。做為系統前端介面、下單畫面最常顯示的名稱。如：台積電。
 - `stock_full_name_zh`：中文全名。記錄公司完整登記名稱，用於詳細資訊頁或正式報表。如：台灣積體電路製造。
 - `stock_name_en` / `stock_full_name_en`：英文簡稱與全名。保留系統未來雙語化之擴充性。
 - `market_type`：上市別。記錄該標的所屬市場 (上市 / 上櫃 / 興櫃)。
 - `sector_name`：產業名。紀錄股票所屬產業。如：半導體業。
 - `listing_date`：最近上市日。做為時光機引擎的重要防呆依據：若玩家將時光機回溯至此日期之前，該檔股票將處於「尚未上市」狀態，禁止任何委託與報價查詢。
 - `par_value`：股票面額。記錄發行面額 (台股現已實施彈性面額，並非全為 10 元)，提供進階財報分析所需之基礎數據。
 - `phone` / `address` / `website`：公司聯絡資訊 (電話、地址、網站)。做為玩家研究該檔股票時的輔助背景資訊。
2. 歷史日線行情 (DailyPrice)：本資料表儲存台股各檔股票於歷史交易日之核心報價與市場狀態，作為判定委託單是否成交之唯一歷史依據，包含：
- `stock_id`：股票代碼 (Foreign Key)。對應至 Stock 表，與 `trade_date` 欄位共同組成此表之複合主鍵。
 - `trade_date`：交易日期 (Partial Key)。記錄該筆報價之所屬日期。
 - `open` / `high` / `low` / `close`：開高低收價格。分別代表該交易日的開盤價、最高價、最低價與收盤價。
 - `volume`：當日成交量 (張數)。作為市場流動性之真實指標，未來可供系統實作「玩家單筆委託量不得超過當日市場總量之一定比例」的進階防呆機制。
 - `ref_price`：當日開盤參考價。通常為前一交易日之收盤價；若遇除權息或面額變更等重大事件，此價格將直接反映扣除股息或分割後之基準價，供前端正確繪製 K 線跳空缺口。

- `limit_up / limit_down`：當日漲停價與當日跌停價。
 - `is_attention / is_disposition / is_full_delivery`：交易異常個股 (注意股、處置股、全額交割股)。採用布林值 (Boolean) 記錄。
3. 暫停交易日期 (`SuspensionDate`)：記錄台股各檔股票之暫停交易日期與原因，作為時光機引擎判定該檔股票是否可交易的防呆依據，包含：
- `stock_id`：股票代碼 (Foreign Key)。對應至 `Stock` 表，與 `suspensio_start_date` 欄位共同組成此表之複合主鍵。
 - `suspend_start_date`：暫停交易開始日期 (Partial Key)。記錄該檔股票開始暫停交易的日期。
 - `resume_date`：恢復交易日期。記錄該檔股票恢復交易的日期。如果是加權指數，則為空值。
 - `reason`：暫停交易原因。記錄該檔股票被暫停交易的原因，如：重大訊息公告、財務報表異常、減資、國定假日或颱風天等。

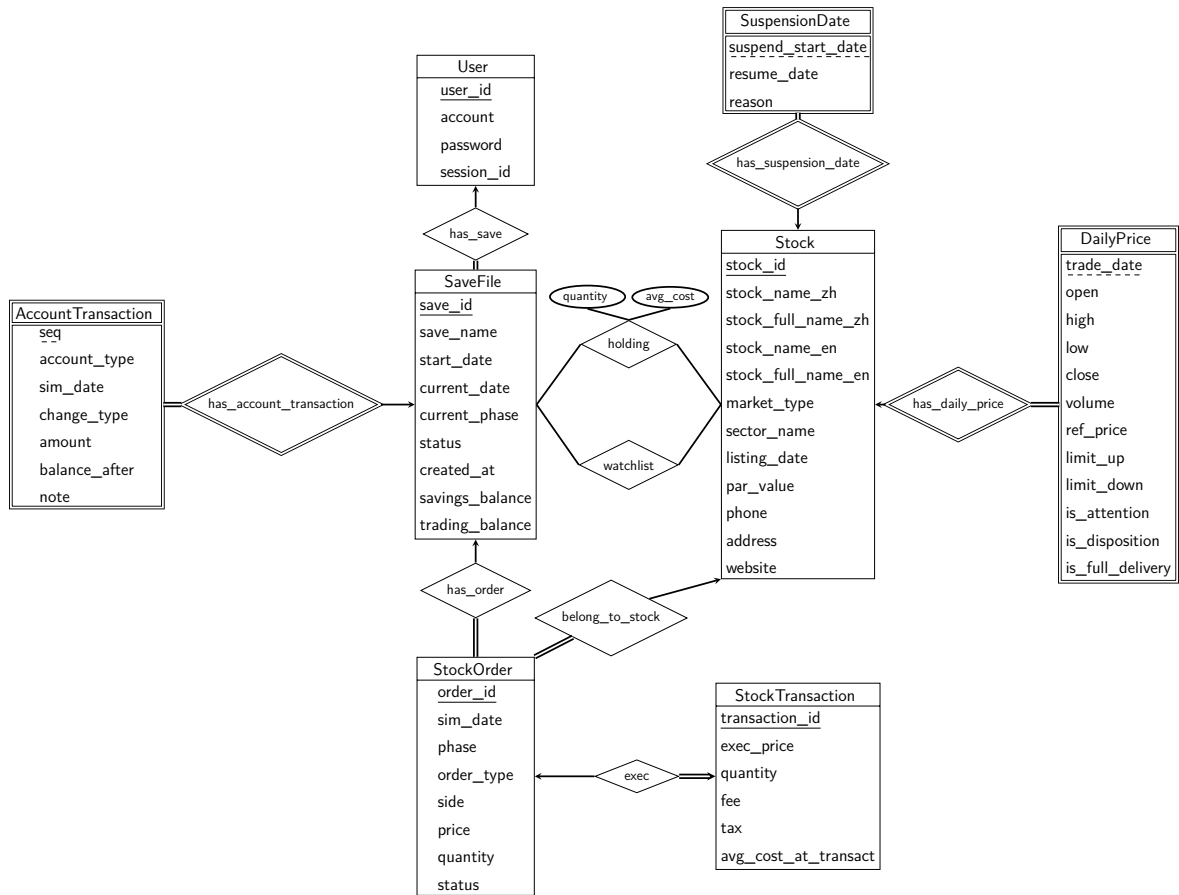
1.3 模擬存檔相關資料

1. 時光機存檔 (`SaveFile`)：作為系統的核心狀態機 (State Machine)，負責控管玩家該次模擬投資的進程、虛擬時間軸與資產隔離狀態，包含：
- `save_id`：存檔唯一識別碼 (Primary Key)。
 - `user_id`：所屬玩家識別碼 (Foreign Key)。具備全部參與 (NOT NULL) 約束，確保每份存檔必有歸屬玩家。
 - `save_name`：存檔名稱。供玩家辨識不同次模擬挑戰的自訂名稱。
 - `start_date`：時光機起點日期。記錄玩家設定的初始歷史日期。
 - `current_date`：當前虛擬日期。隨著玩家點擊推進時光而更新，決定撮合引擎所參考的歷史報價基準。
 - `current_phase`：當前交易階段。以列舉值 (ENUM：盤前/盤中/盤後/收市) 嚴格定義，作為前端畫面鎖定與後端撮合邏輯切換的防呆依據。
 - `status`：存檔狀態。記錄該次模擬為「進行中」或「已破產/已結束」。
 - `savings_balance`：存款戶餘額。代表玩家的閒置銀行本金。

- trading_balance：交割戶餘額。專門用於股票買賣交割與扣收稅費，實現銀行與證券資金隔離的真實情境。
2. 帳戶金流明細 (AccountTransaction)：採用會計流水帳 (Ledger) 概念，完整追蹤存檔內所有帳戶內資金資金異動軌跡，包含：
- save_id：所屬存檔識別碼 (Foreign Key)。
 - seq：存檔內明細序號。做為弱實體的部分鍵 (Partial Key)，標示該存檔的第 N 筆金流。(備註：實務上會與 save_id 共同設定為 Unique 約束防呆)
 - account_type：帳戶類型。標示此筆金流發生於「存款戶」或「交割戶」。
 - sim_date：虛擬發生日期。對應金流發生時的時光機當前日期。
 - change_type：異動類型。記錄造成資金變動的原因 (如：資金轉入、資金轉出、買進交割扣款、賣出交割入帳)。
 - amount：異動金額。紀錄單次操作的絕對數值。
 - balance_after：異動後餘額。記錄該筆金流完成當下的帳戶剩餘總額，便於快速繪製歷史資產曲線，避免耗能的重新加總運算。
 - note：備註說明。提供系統在記錄特殊事件時留下文字說明的彈性欄位。
3. 委託單 (StockOrder)：記錄玩家發出的交易意願與指令，並嚴格控管該委託的生命週期，包含：
- order_id：委託單唯一識別碼 (Primary Key)。
 - save_id：所屬存檔識別碼 (Foreign Key)。標示此張單屬於哪一個時光機存檔。
 - stock_id：委託標的代碼 (Foreign Key)。對應至真實股票代碼。
 - sim_date：委託虛擬日期。記錄玩家按下下單鍵時，時光機所在的當前日期。
 - phase：交易階段 (盤前/盤中/盤後)。
 - order_type：委託單類型。列舉值 (限價/市價)，決定撮合引擎比對價格的邏輯。
 - side：買賣方向。列舉值 (買/賣)。
 - price：委託價格。若為限價單則記錄玩家指定之價格；若為市價單則允許為空值，交由系統搓合。

- quantity：委託張數。需大於 0。
 - status：委託單狀態。列舉值 (待成交/部分成交/已成交/已撤銷/已失效)，隨撮合結果更新。
4. 股票交易紀錄 (StockTransaction)：記錄系統撮合引擎實際執行的交易結果，為玩家扣款與增加持股的絕對依據，包含：
- transaction_id：成交明細唯一識別碼 (Primary Key)。
 - order_id：對應之委託單識別碼 (Foreign Key)。
 - exec_price：實際成交價。系統依據歷史日線 (DailyPrice) 判定後得出的最終成交價格。
 - quantity：成交張數。
 - fee：券商手續費。記錄單筆成交當下結算的整數手續費。
 - tax：證交稅。記錄賣出時依法徵收的整數交易稅 (若為買進則為 0)。
 - avg_cost_at_transact：成交當下平均成本快照。保留玩家在該筆交易發生瞬間的歷史成本，便於未來回溯查詢。
5. 持股 (Holding)：多對多之中介資料表，記錄特定存檔當下的持股狀況，包含：
- save_id：所屬存檔識別碼 (Foreign Key)。
 - stock_id：持有標的代碼 (Foreign Key)。與 save_id 共同組成複合主鍵。
 - quantity：持有張數。隨買賣成交動態增減。
 - avg_cost：加權平均成本。為本系統結算已實現損益之核心計算依據。
6. 自選股 (Watchlist)：多對多之中介資料表，記錄特定存檔進程中，玩家關注之股票清單，包含：
- save_id：所屬存檔識別碼 (Foreign Key)。對應至 SaveFile 表。確保每個存檔擁有獨立的追蹤清單。
 - stock_id：追蹤標的代碼 (Foreign Key)。與 save_id 共同組成複合主鍵，確保單一存檔內不出現重複追蹤之標的。

2 ER Diagram



3 設計概念與限制說明

1. **has_save**: 記錄使用者擁有哪些時光機存檔。使用者 (User) 對時光機存檔 (SaveFile) 為一對多，並且時光機存檔 (SaveFile) 一定對應到唯一一個使用者 (User)。
2. **has_account_transaction**: 時光機存檔 (SaveFile) 對帳戶金流明細 (AccountTransaction) 為一對多，並且帳戶金流明細 (AccountTransaction) 一定對應到唯一一個時光機存檔 (SaveFile)。
3. **has_order**: 記錄時光機存檔中擁有哪些委託單。時光機存檔 (SaveFile) 對委託單 (StockOrder) 為一對多，並且委託單 (StockOrder) 一定對應到唯一一個時光機存檔 (SaveFile)。
4. **exec**: 委託單 (StockOrder) 對股票交易紀錄 (StockTransaction) 為一對一，並

且股票交易紀錄 (StockTransaction) 一定對應到唯一一個委託單 (StockOrder)。

5. `belong_to_stock`: 委託單 (StockOrder) 對股票基本面 (Stock) 為多對一，並且委託單 (StockOrder) 一定對應到唯一一個股票基本面 (Stock)。
6. `holding`: 紀錄時光機存檔中有哪些股票。時光機存檔 (SaveFile) 對股票基本面 (Stock) 為多對多。同時也記錄股票數量 (quantity) 和平均價。
7. `watchlist`: 紀錄時光機存檔中關注的股票有哪些。時光機存檔 (SaveFile) 對股票基本面 (Stock) 為多對多。
8. `has_daily_price`: 股票基本面 (Stock) 對歷史日線行情 (DailyPrice) 為一對多，並且歷史日線行情 (DailyPrice) 一定對應到唯一一個股票基本面 (Stock)。
9. `has_suspension_date`: 股票基本面 (Stock) 對暫停交易日期 (SuspensionDate) 為一對多，並且停牌日期 (SuspensionDate) 一定對應到唯一一個股票基本面 (Stock)。

4 其他設計考量

1. 或許 SaveFile 可做為 weak entity 依賴於 User。

若這樣設計：若將 SaveFile 作為 weak entity，且它又被許多 entity 依賴，會導致所有依賴於 SaveFile 的 entities 都需要 `user_id`、`save_id` 視為 PK。

我們的決策：我們認為這樣操作起來較為麻煩，因此將 SaveFile 作為 strong entity。

2. 目前委託單 (StockOrder) 與成交紀錄 (StockTransaction) 是一對一關係，但真實交易中，一筆委託單可能會分成多次成交，因此若想提高模擬真實性，可能需要改成一對多設計。

本系統為簡化交易邏輯，一筆委託單只會有全部成交和全部不成交這兩種可能，因此維持一對一關係。

A 附錄：Report 1

C 附錄：Report 3

資料庫系統期末專題 Report 3

錢錢錢市—股票模擬系統

第八組

41347001S 楊凱臣 41347009S 周聖詠

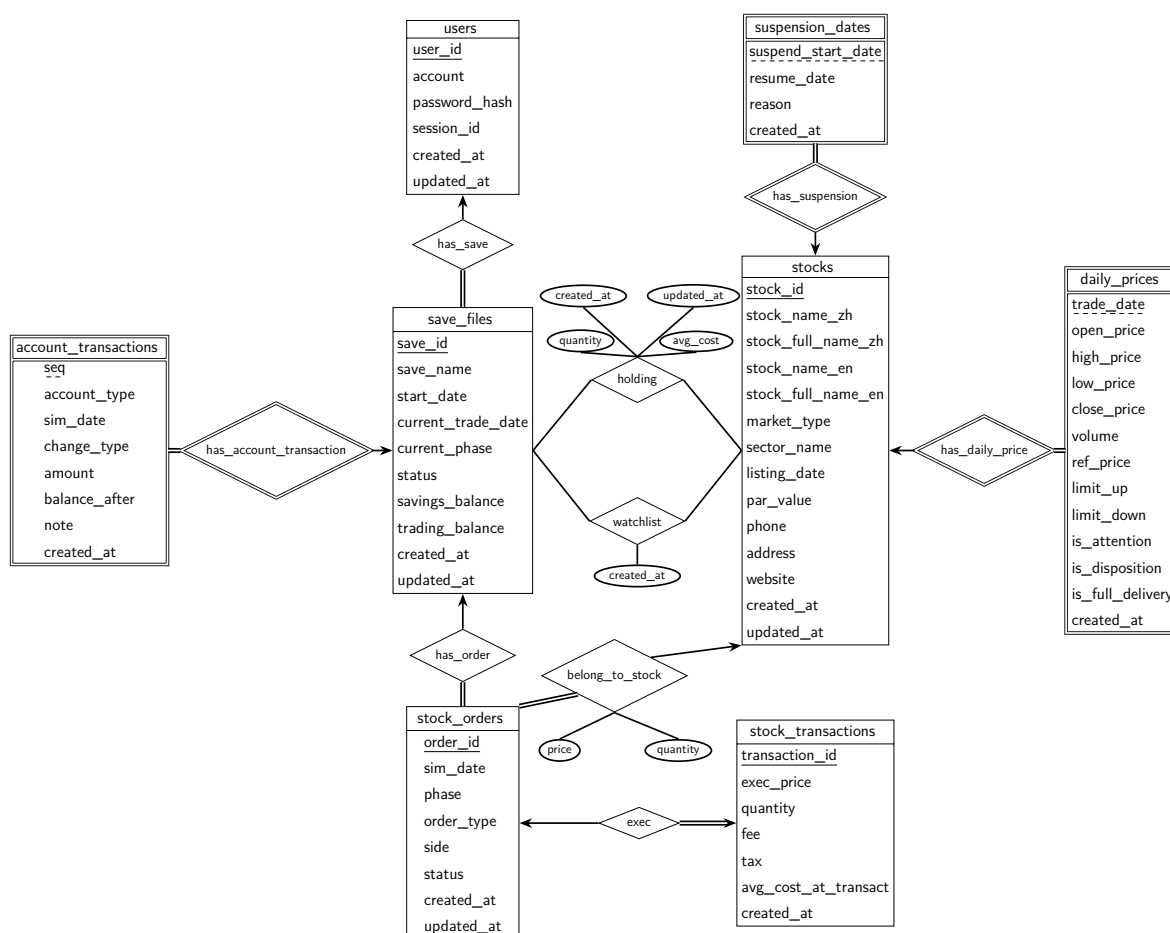
41347011S 許育綸 41347041S 盧人豪

1 Report 2 修改後之 E-R diagram 及說明

1.1 修改前之 E-R diagram

為方便對照，將 Report 2 的 E-R diagram 放於附錄中供參照。

1.2 修改後之 E-R diagram



1.3 說明

1.3.1 回應老師的提問

1. price 與 quantity 的位置。

我們同意老師的建議，將 price 與 quantity 從 StockOrder 的屬性改為 belong_to_stock 關聯的屬性，因為這兩個屬性是描述委託單與股票之間的關係，而非委託單本身的固有屬性。

2. 如果一筆 transation 只有一個 stock 為何有 avg_cost_at_transact？

在此系統架構中，stock_orders（委託單）與 stock_transactions（股票交易紀錄）雖為 1 對 1 關聯，但兩者肩負的「業務職責」完全不同。

- stock_orders 記錄的是「單次行為」：它只在乎「玩家在什麼時間，用多少錢（price）下單了多少張股票」。它是一張單純的收據。
- stock_transactions 記錄的是「財務結果」：它必須承載會計結算的意義。在台股的「移動平均成本法」下，一筆「賣出」訂單的獲利，不能只看這張訂單本身，還必須參考過去所有買進訂單累積下來的平均成本。因此，avg_cost_at_transact 負責補足持股（holding）所缺乏的「歷史持股成本」脈絡。

stock_transactions 作為一個歷史紀錄用，以便之後欲開發回朔功能時更便於回朔。

1.4 修改

1.4.1 修改 Table 命名規則

為了保持資料表命名的一致性與清晰性，我們將所有資料表的名稱統一改為複數形式，並使用底線分隔單字，以符合一般資料庫命名慣例。

1.4.2 欄位名稱

DailyPrice 的 open、high、low、close 欄位名稱修改為 open_price、high_price、low_price、close_price，以更明確的表達欄位內容。User 的 password 欄位名稱修改為 password_hash，以強調該欄位存儲的是密碼的雜湊值，而非明文密碼。

1.4.3 新增時間戳記欄位

為了提升系統的維運追蹤能力、資料除錯效率，以及支援前端介面的功能展示，我們在所有資料表中全面導入了時間戳記 (Timestamp) 欄位。依據資料的生命週期與變動特性，具體的設計邏輯分為以下兩類：

- 狀態會變動的資料表 (新增 `created_at` 與 `updated_at`)：

針對資料建立後，其內容或狀態會隨著業務邏輯持續更新的實體，例如 `users`、`stocks`、`save_files`、`stock_orders` 與 `holdings`。這能幫助系統精確掌握資料的初始化時間與最後一次修改時間 (例如追蹤一筆委託單狀態從 `PENDING` 轉為 `FILLED` 的時間點)。

- 歷史紀錄與狀態單一的資料表 (僅新增 `created_at`)：

針對屬於唯讀日誌性質，或者只有「新增／刪除」而沒有「修改」行為的資料表，我們僅保留建立時間以節省空間。此類包含 `account_transactions`、`daily_prices`、`stock_transactions`、`suspension_dates` 以及 `watchlists`。

1.4.4 欄位名稱與保留字相同之修正

在 `save_files` 表中，將 `current_date` 欄位名稱修改為 `current_trade_date`，以避免與 SQL 保留字 `CURRENT_DATE` 發生衝突，並更明確地表達該欄位的用途。

2 E-R diagram 轉成 Relational Table 之 DDL 的形式

2.1 使用者憑證表 (users)

```

1 CREATE TABLE users (
2     user_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
3     account VARCHAR(50) NOT NULL,
4     password_hash VARCHAR(255) NOT NULL,
5     session_id CHAR(36) NULL,
6     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
7     updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
8     CURRENT_TIMESTAMP,
9     PRIMARY KEY (user_id),
10    UNIQUE KEY uq_users_account (account),
11    UNIQUE KEY uq_users_session_id (session_id)
12 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

```

此資料表負責儲存玩家登入帳戶的憑證與登入狀態資訊，用以進行身分驗證與單一登入防呆。

欄位名稱	資料型態	欄位說明
user_id	BIGINT UNSIGNED	使用者唯一識別碼 (Primary Key，自動遞增)
account	VARCHAR(50)	登入帳號 (Unique Key)
password_hash	VARCHAR(255)	密碼的安全雜湊值 (防範明碼外洩)
session_id	CHAR(36)	登入狀態憑證 Token (Unique Key，防範異地登入)
created_at	TIMESTAMP	帳戶註冊建立時間
updated_at	TIMESTAMP	帳戶狀態最後更新時間

外鍵與約束限制 (Constraints)

- PRIMARY KEY：user_id。
- UNIQUE KEY：
 - uq_users_account (account)：確保帳號不重複。
 - uq_users_session_id (session_id)：確保同一時間只有一處登入憑證有效（單一登入原則，允許為 NULL）。

2.2 股票基本面表 (stocks)

```

1 CREATE TABLE stocks (
2     stock_id VARCHAR(16) NOT NULL,
3     stock_name_zh VARCHAR(64) NOT NULL,
4     stock_full_name_zh VARCHAR(128) NOT NULL,
5     stock_name_en VARCHAR(64) NULL,
6     stock_full_name_en VARCHAR(128) NULL,
7     market_type ENUM('TSE', 'OTC', 'EMERGING', 'INDEX') NOT NULL,
8     sector_name VARCHAR(64) NULL,
9     listing_date DATE NULL,
10    par_value DECIMAL(10, 2) NULL,
11    phone VARCHAR(32) NULL,
12    address VARCHAR(255) NULL,

```

```

13     website VARCHAR(255) NULL,
14     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
15     updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
16     CURRENT_TIMESTAMP,
17     PRIMARY KEY (stock_id),
18     KEY idx_stocks_market_type (market_type),
19     KEY idx_stocks_sector_name (sector_name)
20 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

```

此資料表儲存台股各檔股票與指數之靜態基本面資訊，作為唯讀的字典檔。其上市日期 `listing_date` 亦提供進行交易日期的上市資格校驗。

欄位名稱	資料型態	欄位說明
<code>stock_id</code>	VARCHAR(16)	股票或指數代碼 (Primary Key，如 2330)
<code>stock_name_zh</code>	VARCHAR(64)	中文簡稱 (如台積電)
<code>stock_full_name_zh</code>	VARCHAR(128)	中文全名 (如台灣積體電路製造)
<code>stock_name_en</code>	VARCHAR(64)	英文簡稱 (可為 NULL)
<code>stock_full_name_en</code>	VARCHAR(128)	英文全名 (可為 NULL)
<code>market_type</code>	ENUM(...)	市場別 (TSE: 上市, OTC: 上櫃, EMERGING: 興櫃, INDEX: 指數)
<code>sector_name</code>	VARCHAR(64)	產業類別名稱 (如半導體業，可為 NULL)
<code>listing_date</code>	DATE	最近一次上市/發行日期 (可為 NULL)
<code>par_value</code>	DECIMAL(10, 2)	股票每股面額 (如 10.00，可為 NULL)
<code>phone</code>	VARCHAR(32)	公司聯絡電話 (可為 NULL)
<code>address</code>	VARCHAR(255)	公司地址 (可為 NULL)
<code>website</code>	VARCHAR(255)	公司官方網站 URL (可為 NULL)
<code>created_at</code>	TIMESTAMP	欄位建立時間
<code>updated_at</code>	TIMESTAMP	欄位最後更新時間

外鍵與約束限制 (Constraints)

- **PRIMARY KEY** : `stock_id`。
- **INDEX** :
 - `idx_stocks_market_type (market_type)` : 加速依市場類型篩選股票之查詢。
 - `idx_stocks_sector_name (sector_name)` : 加速依產業篩選股票之查詢。

2.3 歷史日線行情表 (daily_prices)

```

1 CREATE TABLE daily_prices (
2     stock_id VARCHAR(16) NOT NULL,
3     trade_date DATE NOT NULL,
4     open_price DECIMAL(10, 2) NOT NULL,
5     high_price DECIMAL(10, 2) NOT NULL,
6     low_price DECIMAL(10, 2) NOT NULL,
7     close_price DECIMAL(10, 2) NOT NULL,
8     volume BIGINT UNSIGNED NOT NULL,
9     ref_price DECIMAL(10, 2) NULL,
10    limit_up DECIMAL(10, 2) NULL,
11    limit_down DECIMAL(10, 2) NULL,
12    is_attention BOOLEAN NOT NULL DEFAULT FALSE,
13    is_disposition BOOLEAN NOT NULL DEFAULT FALSE,
14    is_full_delivery BOOLEAN NOT NULL DEFAULT FALSE,
15    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
16    PRIMARY KEY (stock_id, trade_date),
17    KEY idx_daily_prices_trade_date (trade_date),
18    CONSTRAINT fk_daily_prices_stock
19        FOREIGN KEY (stock_id) REFERENCES stocks (stock_id)
20        ON UPDATE CASCADE
21        ON DELETE RESTRICT,
22    CONSTRAINT chk_daily_prices_price_range
23        CHECK (
24            open_price >= 0
25            AND high_price >= 0
26            AND low_price >= 0
27            AND close_price >= 0
28            AND (ref_price IS NULL OR ref_price >= 0)
29            AND (limit_up IS NULL OR limit_up >= 0)
30            AND (limit_down IS NULL OR limit_down >= 0)
31            AND low_price <= high_price
32            AND low_price <= open_price
33            AND low_price <= close_price
34            AND high_price >= open_price
35            AND high_price >= close_price
36            AND (limit_up IS NULL OR limit_down IS NULL OR limit_up >=
limit_down)
37        ),
38    CONSTRAINT chk_daily_prices_non_negative_volume

```



```

39      CHECK (volume >= 0)
40 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

```

此資料表儲存每檔股票或指數在每個交易日的開盤、最高、最低、收盤價格與成交量等行情，為交易撮合與 K 線圖繪製之核心依據。

欄位名稱	資料型態	欄位說明
stock_id	VARCHAR(16)	股票或指數代碼 (Primary Key , Foreign Key)
trade_date	DATE	交易日期 (Primary Key)
open_price	DECIMAL(10, 2)	今日開盤價
high_price	DECIMAL(10, 2)	今日最高價
low_price	DECIMAL(10, 2)	今日最低價
close_price	DECIMAL(10, 2)	今日收盤價
volume	BIGINT UNSIGNED	當日成交量 (單位：股)
ref_price	DECIMAL(10, 2)	當日開盤參考價 (可為 NULL)
limit_up	DECIMAL(10, 2)	當日漲停價 (可為 NULL)
limit_down	DECIMAL(10, 2)	當日跌停價 (可為 NULL)
is_attention	BOOLEAN	是否為注意股 (預設為 FALSE)
is_disposition	BOOLEAN	是否為處置股 (預設為 FALSE)
is_full_delivery	BOOLEAN	是否為全額交割股 (預設為 FALSE)
created_at	TIMESTAMP	欄位建立時間

外鍵與約束限制 (Constraints)

- PRIMARY KEY : (stock_id, trade_date)。
- FOREIGN KEY :
 - fk_daily_prices_stock : stock_id 參考 stocks(stock_id)，採 ON UPDATE CASCADE ON DELETE RESTRICT 以維護關聯完整性。
- INDEX :
 - idx_daily_prices_trade_date (trade_date) : 加速跨股票查詢特定交易日報價之效能。

- CHECK：

- chk_daily_prices_price_range：確保所有價格欄位均為非負數，且滿足合理價格區間限制（最低價 $\leq$ 開/收/最高價，最高價 $\geq$ 開/收價，以及漲停價 $\geq$ 跌停價）。
- chk_daily_prices_non_negative_volume：確保成交量大於或等於 0。

2.4 暫停交易日期表 (suspension_dates)

```

1 CREATE TABLE suspension_dates (
2     stock_id VARCHAR(16) NOT NULL,
3     suspend_start_date DATE NOT NULL,
4     resume_date DATE NULL,
5     reason VARCHAR(255) NOT NULL,
6     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
7     PRIMARY KEY (stock_id, suspend_start_date),
8     KEY idx_suspension_dates_resume_date (resume_date),
9     CONSTRAINT fk_suspension_dates_stock
10         FOREIGN KEY (stock_id) REFERENCES stocks (stock_id)
11         ON UPDATE CASCADE
12         ON DELETE RESTRICT,
13     CONSTRAINT chk_suspension_dates_order
14         CHECK (resume_date IS NULL OR resume_date >= suspend_start_date)
15 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

```

欄位名稱	資料型態	欄位說明
stock_id	VARCHAR(16)	股票代碼 (Primary Key , Foreign Key)
suspend_start_date	DATE	暫停交易開始日期 (Primary Key)
resume_date	DATE	恢復交易日期 (可為 NULL，代表尚未復牌或為指數型商品)
reason	VARCHAR(255)	暫停交易原因 (如減資、颱風天、重大訊息等)
created_at	TIMESTAMP	欄位建立時間

外鍵與約束限制 (Constraints)

- PRIMARY KEY：(stock_id, suspend_start_date)。

- FOREIGN KEY：

- fk_suspension_dates_stock：stock_id 參考 stocks(stock_id)，採 ON UPDATE CASCADE ON DELETE RESTRICT。

- INDEX：

- idx_suspension_dates_resume_date (resume_date)：加速篩選並查詢已復牌股票之效能。

- CHECK：

- chk_suspension_dates_order：確保復牌日期大於或等於停牌開始日期 (resume_date >= suspend_start_date，若為 NULL 則不限)。

2.5 模擬存檔表 (save_files)

```

1 CREATE TABLE save_files (
2     save_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
3     user_id BIGINT UNSIGNED NOT NULL,
4     save_name VARCHAR(100) NOT NULL,
5     start_date DATE NOT NULL,
6     current_trade_date DATE NOT NULL,
7     current_phase ENUM('PRE_MARKET', 'INTRADAY', 'POST_MARKET', 'CLOSED')
8     NOT NULL,
9     status ENUM('ACTIVE', 'BANKRUPT', 'FINISHED') NOT NULL DEFAULT 'ACTIVE',
10     savings_balance DECIMAL(18, 2) NOT NULL DEFAULT 0.00,
11     trading_balance DECIMAL(18, 2) NOT NULL DEFAULT 0.00,
12     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
13     updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
14     CURRENT_TIMESTAMP,
15     PRIMARY KEY (save_id),
16     UNIQUE KEY uq_save_files_user_save_name (user_id, save_name),
17     KEY idx_save_files_user_id (user_id),
18     KEY idx_save_files_user_status (user_id, status),
19     KEY idx_save_files_status (status),
20     CONSTRAINT fk_save_files_user
21     FOREIGN KEY (user_id) REFERENCES users (user_id)
22     ON UPDATE CASCADE

```

```

21         ON DELETE RESTRICT,
22     CONSTRAINT chk_save_files_balances
23         CHECK (savings_balance >= 0 AND trading_balance >= 0),
24     CONSTRAINT chk_save_files_dates
25         CHECK (current_trade_date >= start_date)
26 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

```

此資料表作為模擬系統的核心狀態機，負責控管個別玩家的模擬進度、虛擬時間軸、當前交易階段與雙帳戶餘額。

欄位名稱	資料型態	欄位說明
save_id	BIGINT UNSIGNED	存檔唯一識別碼 (Primary Key，自動遞增)
user_id	BIGINT UNSIGNED	所屬玩家識別碼 (Foreign Key)
save_name	VARCHAR(100)	存檔自訂名稱
start_date	DATE	時光機模擬初始歷史日期
current_trade_date	DATE	當前虛擬交易日期
current_phase	ENUM(...)	當前交易階段 (PRE_MARKET, INTRA-DAY, POST_MARKET, CLOSED)
status	ENUM(...)	存檔生命週期狀態 (ACTIVE, BANKRUPT, FINISHED)
savings_balance	DECIMAL(18, 2)	存款戶餘額 (預設為 0.00)
trading_balance	DECIMAL(18, 2)	交割戶餘額 (預設為 0.00)
created_at	TIMESTAMP	存檔建立時間
updated_at	TIMESTAMP	存檔狀態最後更新時間

外鍵與約束限制 (Constraints)

- **PRIMARY KEY**：save_id。
- **UNIQUE KEY**：
 - uq_save_files_user_save_name(user_id, save_name)：確保同一個玩家無法建立同名的模擬存檔。
- **FOREIGN KEY**：
 - fk_save_files_user：user_id 參考 users(user_id)，採 ON UPDATE CASCADE ON DELETE RESTRICT。

- INDEX：

- idx_save_files_user_id (user_id)：加速查詢特定玩家的所有存檔。
- idx_save_files_user_status (user_id, status)：加速查詢與計算玩家目前進行中的存檔數量。
- idx_save_files_status (status)：加速依存檔狀態進行全局數據統計。

- CHECK：

- chk_save_files_balances：確保存款戶與交割戶之餘額皆大於或等於 0，避免產生非法的負餘額。
- chk_save_files_dates：確保當前虛擬交易日期大於或等於初始歷史日期 (current_trade_date >= start_date)。

2.6 帳戶金流明細表 (account_transactions)

```

1 CREATE TABLE account_transactions (
2     save_id BIGINT UNSIGNED NOT NULL,
3     seq INT UNSIGNED NOT NULL,
4     account_type ENUM('SAVINGS', 'TRADING') NOT NULL,
5     sim_date DATE NOT NULL,
6     change_type VARCHAR(50) NOT NULL,
7     amount DECIMAL(18, 2) NOT NULL,
8     balance_after DECIMAL(18, 2) NOT NULL,
9     note VARCHAR(255) NULL,
10    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
11    PRIMARY KEY (save_id, seq),
12    KEY idx_account_transactions_sim_date (sim_date),
13    KEY idx_account_transactions_change_type (change_type),
14    CONSTRAINT fk_account_transactions_save
15        FOREIGN KEY (save_id) REFERENCES save_files (save_id)
16        ON UPDATE CASCADE
17        ON DELETE CASCADE,
18    CONSTRAINT chk_account_transactions_seq
19        CHECK (seq > 0),
20    CONSTRAINT chk_account_transactions_amount
21        CHECK (amount >= 0),
22    CONSTRAINT chk_account_transactions_balance_after

```

```

23      CHECK (balance_after >= 0)
24 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

```

此資料表採用會計流水帳的概念，完整記錄玩家模擬存檔中存款戶與交割戶的每一筆資金異動軌跡，用以進行歷史金流追蹤與繪製資產曲線。

欄位名稱	資料型態	欄位說明
save_id	BIGINT UNSIGNED	所屬存檔識別碼 (Primary Key , Foreign Key)
seq	INT UNSIGNED	存檔內明細流水號 (Primary Key , 由 1 開始遞增)
account_type	ENUM(...)	帳戶類型 (SAVINGS: 存款戶, TRADING: 交割戶)
sim_date	DATE	金流發生的虛擬日期
change_type	VARCHAR(50)	異動類型 (如資金轉帳、交易扣款、交易入帳等)
amount	DECIMAL(18, 2)	金流異動金額
balance_after	DECIMAL(18, 2)	異動後之帳戶餘額快照
note	VARCHAR(255)	額外備註說明 (可為 NULL)
created_at	TIMESTAMP	明細記錄建立時間

外鍵與約束限制 (Constraints)

- **PRIMARY KEY** : (save_id, seq) 。
- **FOREIGN KEY** :
 - fk_account_transactions_save : save_id 參考 save_files(save_id) , 採 ON UPDATE CASCADE ON DELETE CASCADE , 確保存存檔刪除時明細同步清除。
- **INDEX** :
 - idx_account_transactions_sim_date (sim_date) : 加速特定模擬日期之金流明細查詢。
 - idx_account_transactions_change_type (change_type) : 加速特定交易類型的查詢。

- CHECK：

- chk_account_transactions_seq：確保流水號 seq 大於 0。
- chk_account_transactions_amount：確保異動金額大於或等於 0。
- chk_account_transactions_balance_after：確保異動後之餘額快照大於或等於 0。

2.7 股票委託單表 (stock_orders)

```

1 CREATE TABLE stock_orders (
2     order_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
3     save_id BIGINT UNSIGNED NOT NULL,
4     stock_id VARCHAR(16) NOT NULL,
5     sim_date DATE NOT NULL,
6     phase ENUM('PRE_MARKET', 'INTRADAY', 'POST_MARKET') NOT NULL,
7     order_type ENUM('LIMIT', 'MARKET') NOT NULL,
8     side ENUM('BUY', 'SELL') NOT NULL,
9     price DECIMAL(10, 2) NULL,
10    quantity INT UNSIGNED NOT NULL,
11    status ENUM('PENDING', 'FILLED', 'CANCELED', 'EXPIRED', 'REJECTED')
    NOT NULL DEFAULT 'PENDING',
12    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
13    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
    CURRENT_TIMESTAMP,
14    PRIMARY KEY (order_id),
15    KEY idx_stock_orders_save_id (save_id),
16    KEY idx_stock_orders_stock_id (stock_id),
17    KEY idx_stock_orders_sim_date (sim_date),
18    KEY idx_stock_orders_status (status),
19    CONSTRAINT fk_stock_orders_save
20        FOREIGN KEY (save_id) REFERENCES save_files (save_id)
21        ON UPDATE CASCADE
22        ON DELETE CASCADE,
23    CONSTRAINT fk_stock_orders_stock
24        FOREIGN KEY (stock_id) REFERENCES stocks (stock_id)
25        ON UPDATE CASCADE
26        ON DELETE RESTRICT,
27    CONSTRAINT chk_stock_orders_quantity
28        CHECK (quantity > 0),

```

```

29  CONSTRAINT chk_stock_orders_price
30  CHECK (
31      (order_type = 'LIMIT' AND price IS NOT NULL AND price > 0)
32      OR (order_type = 'MARKET' AND price IS NULL)
33  )
34 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

```

此資料表記錄玩家在特定模擬存檔中發出的買賣委託單，包含限價單、市價單與定價委託，為撮合引擎的運算對象。

欄位名稱	資料型態	欄位說明
order_id	BIGINT UNSIGNED	委託單唯一識別碼 (Primary Key，自動遞增)
save_id	BIGINT UNSIGNED	所屬存檔識別碼 (Foreign Key)
stock_id	VARCHAR(16)	委託標的代碼 (Foreign Key)
sim_date	DATE	委託時的虛擬日期
phase	ENUM(...)	委託交易階段 (PRE_MARKET, INTRA-DAY, POST_MARKET)
order_type	ENUM(...)	委託類型 (LIMIT: 限價單, MARKET: 市價單)
side	ENUM(...)	買賣方向 (BUY: 買進, SELL: 賣出)
price	DECIMAL(10, 2)	限價單之指定價格 (市價單則為 NULL)
quantity	INT UNSIGNED	委託股數 (單位：股，下單最小限制為 1000 股)
status	ENUM(...)	委託單狀態 (PENDING: 待撮合, FILLED: 已成交, CANCELED: 撤單, EXPIRED: 逾期失效, REJECTED: 拒絕下單)
created_at	TIMESTAMP	委託單建立時間
updated_at	TIMESTAMP	委託單狀態最後更新時間

外鍵與約束限制 (Constraints)

- PRIMARY KEY：order_id。
- FOREIGN KEY：
 - fk_stock_orders_save：save_id 參考 save_files(save_id)，採 ON UPDATE CASCADE ON DELETE CASCADE。

- `fk_stock_orders_stock` : `stock_id` 參考 `stocks(stock_id)` ,
採 `ON UPDATE CASCADE ON DELETE RESTRICT` 。

- INDEX :

- `idx_stock_orders_save_id (save_id)` : 加速查詢特定存檔的委託單。
- `idx_stock_orders_stock_id (stock_id)` : 加速查詢特定股票的委託紀錄。
- `idx_stock_orders_sim_date (sim_date)` : 加速依模擬日期篩選委託單。
- `idx_stock_orders_status (status)` : 加速篩選未成交或已成交的委託單以進行撮合。

- CHECK :

- `chk_stock_orders_quantity` : 確保委託股數大於 0 。
- `chk_stock_orders_price` : 確保限價單必須指定大於 0 的價格；市價單之價格欄位必須為空值 (NULL) 。

2.8 股票交易紀錄表 (stock_transactions)

```

1 CREATE TABLE stock_transactions (
2     transaction_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
3     order_id BIGINT UNSIGNED NOT NULL,
4     exec_price DECIMAL(10, 2) NOT NULL,
5     quantity INT UNSIGNED NOT NULL,
6     fee DECIMAL(10, 2) NOT NULL DEFAULT 0.00,
7     tax DECIMAL(10, 2) NOT NULL DEFAULT 0.00,
8     avg_cost_at_transact DECIMAL(10, 4) NULL,
9     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
10    PRIMARY KEY (transaction_id),
11    UNIQUE KEY uq_stock_transactions_order_id (order_id),
12    CONSTRAINT fk_stock_transactions_order
13        FOREIGN KEY (order_id) REFERENCES stock_orders (order_id)
14        ON UPDATE CASCADE
15        ON DELETE CASCADE,
16    CONSTRAINT chk_stock_transactions_quantity
17        CHECK (quantity > 0),
18    CONSTRAINT chk_stock_transactions_amounts

```

```

19      CHECK (exec_price > 0 AND fee >= 0 AND tax >= 0 AND (
      avg_cost_at_transact IS NULL OR avg_cost_at_transact >= 0))
20 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

```

欄位名稱	資料型態	欄位說明
transaction_id	BIGINT UNSIGNED	交易明細唯一識別碼 (Primary Key, 自動遞增)
order_id	BIGINT UNSIGNED	對應之委託單識別碼 (Unique Key, Foreign Key)
exec_price	DECIMAL(10, 2)	實際成交單價
quantity	INT UNSIGNED	成交股數 (單位：股)
fee	DECIMAL(10, 2)	券商交易手續費 (買賣雙向收取，最低 20 元)
tax	DECIMAL(10, 2)	國家證券交易稅 (僅在賣出時收取，稅率 0.3%)
avg_cost_at_transact	DECIMAL(10, 4)	成交當下的持股平均成本快照 (可為 NULL)
created_at	TIMESTAMP	交易紀錄建立時間

外鍵與約束限制 (Constraints)

- **PRIMARY KEY** : transaction_id。
- **UNIQUE KEY** :
 - uq_stock_transactions_order_id (order_id) : 確保一筆委託單至多只會對應一筆成交明細，落實 1 對 1 關係。
- **FOREIGN KEY** :
 - fk_stock_transactions_order : order_id 參考 stock_orders(order_id)，採 ON UPDATE CASCADE ON DELETE CASCADE。
- **CHECK** :
 - chk_stock_transactions_quantity : 確保成交股數大於 0。
 - chk_stock_transactions_amounts : 確保成交單價大於 0，手續費、證交稅均大於或等於 0，且若平均成本快照不為空值，則其必須大於或等於 0。

2.9 持股表 (holdings)

```

1 CREATE TABLE holdings (
2     save_id BIGINT UNSIGNED NOT NULL,
3     stock_id VARCHAR(16) NOT NULL,
4     quantity INT UNSIGNED NOT NULL,
5     avg_cost DECIMAL(10, 4) NOT NULL,
6     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
7     updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
8     CURRENT_TIMESTAMP,
9     PRIMARY KEY (save_id, stock_id),
10    KEY idx_holdings_stock_id (stock_id),
11    CONSTRAINT fk_holdings_save
12        FOREIGN KEY (save_id) REFERENCES save_files (save_id)
13        ON UPDATE CASCADE
14        ON DELETE CASCADE,
15    CONSTRAINT fk_holdings_stock
16        FOREIGN KEY (stock_id) REFERENCES stocks (stock_id)
17        ON UPDATE CASCADE
18        ON DELETE RESTRICT,
19    CONSTRAINT chk_holdings_values
20        CHECK (quantity >= 0 AND avg_cost >= 0)
21 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

```

此資料表作為多對多之中介表，用以追蹤玩家在各模擬存檔中的股票庫存數量與加權平均成本，為計算帳面市值與未實現損益的核心來源。

欄位名稱	資料型態	欄位說明
save_id	BIGINT UNSIGNED	所屬存檔識別碼 (Primary Key , Foreign Key)
stock_id	VARCHAR(16)	持有標的代碼 (Primary Key , Foreign Key)
quantity	INT UNSIGNED	持有股數 (單位：股，隨成交動態增減)
avg_cost	DECIMAL(10, 4)	加權平均成本 (單位：元/股，含交易手續費攤平)
created_at	TIMESTAMP	庫存欄位建立時間
updated_at	TIMESTAMP	庫存數量與成本最後更新時間

外鍵與約束限制 (Constraints)

- PRIMARY KEY : (save_id, stock_id)。

- FOREIGN KEY：

- fk_holdings_save：save_id 參考 save_files(save_id)，
採 ON UPDATE CASCADE ON DELETE CASCADE。
- fk_holdings_stock：stock_id 參考 stocks(stock_id)，
採 ON UPDATE CASCADE ON DELETE RESTRICT。

- INDEX：

- idx_holdings_stock_id (stock_id)：加速跨存檔查詢某檔股票的全局持有分布。

- CHECK：

- chk_holdings_values：確保持有股數 quantity 與平均成本 avg_cost 均大於或等於 0。

2.10 自選股表 (watchlists)

```

1 CREATE TABLE watchlists (
2     save_id BIGINT UNSIGNED NOT NULL,
3     stock_id VARCHAR(16) NOT NULL,
4     created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
5     PRIMARY KEY (save_id, stock_id),
6     KEY idx_watchlists_stock_id (stock_id),
7     CONSTRAINT fk_watchlists_save
8         FOREIGN KEY (save_id) REFERENCES save_files (save_id)
9         ON UPDATE CASCADE
10        ON DELETE CASCADE,
11    CONSTRAINT fk_watchlists_stock
12        FOREIGN KEY (stock_id) REFERENCES stocks (stock_id)
13        ON UPDATE CASCADE
14        ON DELETE RESTRICT
15 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

此資料表作為多對多之中介表，用以追蹤玩家在各模擬存檔中自選關注的股票清單，確保跨存檔自選清單的完全隔離。

欄位名稱	資料型態	欄位說明
save_id	BIGINT UNSIGNED	所屬存檔識別碼 (Primary Key , Foreign Key)
stock_id	VARCHAR(16)	自選關注之股票代碼 (Primary Key , Foreign Key)
created_at	TIMESTAMP	加入自選清單時間

外鍵與約束限制 (Constraints)

- **PRIMARY KEY** : (save_id, stock_id)。
- **FOREIGN KEY** :
 - fk_watchlists_save : save_id 參考 save_files(save_id) , 採 ON UPDATE CASCADE ON DELETE CASCADE。
 - fk_watchlists_stock : stock_id 參考 stocks(stock_id) , 採 ON UPDATE CASCADE ON DELETE RESTRICT。
- **INDEX** :
 - idx_watchlists_stock_id (stock_id) : 加速跨存檔查詢某檔股票被自選追蹤之次數。
- **TRIGGER 約束** :
 - 設有 trg_watchlists_before_insert 觸發器，限制每個存檔最多只能加入 100 筆自選股。

3 系統內容功能詳細敘述

本模擬系統旨在建立一個高度擬真的台灣股市交易沙盒環境。為了使系統兼顧遊玩趣味性與金融交易的嚴謹性，系統功能圍繞於「玩家帳戶管理」、「模擬存檔隔離」、「時光機時間軸控制」、「銀行雙帳戶資產分配」、「盤面行情與技術指標查詢」、「撮合引擎邏輯」以及「持股與損益結算」等七個核心模組。

3.1 系統核心功能模組

1. 帳戶登入與安全驗證

系統為單人模擬器，使用者需先進行註冊與登入。資料庫使用 Bcrypt 演算法安全儲存密碼。此外，為了防範 Race Condition，採用了單一登入 (SSO) 原則：使用者登入後，系統會生成一個安全登入憑證並寫入 session_id，若同帳號於其他瀏覽器再次登入，新憑證將會覆蓋舊有憑證，使舊登入狀態失效。

2. 多重模擬存檔管理

每位使用者可同時建立並管理多個模擬存檔（時光沙盒），每個存檔的時間軸、資產餘額、持股庫存、自選股以及交易紀錄完全獨立，互不干涉。

- **進行中存檔上限：**每位玩家最多可同時擁有 5 個進行中 (ACTIVE) 存檔。
- **總存檔上限：**包含進行中、已破產 (BANKRUPT) 與手動結束封存 (FINISHED)，每位玩家上限為 10 個。
- **防呆限制：**同一玩家帳戶下不可建立同名的模擬存檔。

3. 時間軸控制與資訊隔離

系統以「交易日」為基本時間單位。使用者可自由選擇歷史起點日期（或由系統隨機抽取），並手動點擊推進時間。每個交易日依序切換四個交易階段：盤前 → 盤中 → 盤後 → 收市。

時光機引導之歷史資料揭露需嚴格遵守「時空隔離」原則，玩家僅能查詢當前虛擬日期以前已揭露之 K 線與價格，未推進到的日期或當日未收盤的資訊不得提前顯示，以徹底杜絕未來套利。

4. 銀行系統與雙帳戶資產分配

為模擬真實的證券交易扣款，系統提供雙帳戶機制：

- **存款戶：**用於存放玩家暫不投入股票市場之銀行本金。
- **交割戶：**專門用於股票買賣成交時的款項交割與證券稅費扣收。
- 兩帳戶之間可即時互轉資金，不收任何費用。在正常狀態下，兩帳戶的餘額皆大於或等於 0，交割戶扣款不足即代表模擬破產。

5. 股票查詢與自選股管理

使用者可透過搜尋或市場類別瀏覽股票，並檢視歷史日 K、週 K、月 K 線圖與同週期的成交量圖。玩家可自由疊加常見技術指標。自選股清單隸屬於個別存檔，上限為 100 筆，且同一存檔內同檔股票不可重複加入。

6. 交易規則限制

- **整股交易**：所有委託買賣皆以整股（最低 1000 股）為單位，不接受零股交易。
- **不支援部分成交**：撮合結果僅有全部成交 (FILLED) 或全部不成交 (EXPIRED) 兩種。
- **無放空機制**：本版本不支援融資融券，且送出賣單時系統會先扣除當前階段所有 Pending 賣單，嚴格禁止超額賣出。
- **交易稅費**：買進手續費 0.1425%；賣出手續費 0.1425%，另收證券交易稅 0.3%（單筆手續費不足 20 元以 20 元計）。

7. 持股均價與損益結算

持股採加權移動平均法計算每股平均成本。

- **未實現損益**：以目前階段已知的最新價格計算（盤前為前一日收盤價，盤中為今日開盤價，盤後與收市為今日收盤價）。
- **已實現損益**：於賣出成交時，依據成交價扣除交易稅費，並與持股平均成本（從 avg_cost_at_transact 取得）比對計算。

3.2 撮合引擎之八種交易撮合邏輯

系統的核心撮合引擎依據時間軸階段的切換來執行撮合，其具體運作邏輯如下：

(1) 盤前限價買單

- **下單時機**：玩家於盤前階段 (08:30 - 09:00) 送出限價買單。
- **撮合結算時機**：當玩家點擊推進至盤中階段時執行。
- **成交條件**：限價 $\geq$ 當日開盤價 (open_price)。
- **實際成交價**：當日開盤價 (open_price)。
- **未成交條件**：限價 $<$ 當日開盤價，委託單自動失效 (EXPIRED)。
- **資金交割**：成交後立即從交割戶扣款，若交割戶餘額不足則存檔狀態轉為 BANKRUPT。

(2) 盤前限價賣單

- 下單時機：玩家於盤前階段送出限價賣單。
- 撮合結算時機：當玩家點擊推進至盤中階段時執行。
- 成交條件：限價 $\leq$ 當日開盤價 (open_price)。
- 實際成交價：當日開盤價 (open_price)。
- 未成交條件：限價 $>$ 當日開盤價，委託單自動失效 (EXPIRED)。
- 資金交割：成交後立即將扣除手續費與證交稅後的淨額入帳至交割戶。

(3) 盤中限價買單

- 下單時機：玩家於盤中階段 (09:00 - 13:30) 送出限價買單。
- 撮合結算時機：當玩家點擊推進至盤後階段時執行。
- 成交條件：限價 $\geq$ 當日最低價 (low_price)。
- 實際成交價：玩家指定的限制價格。
- 未成交條件：限價 $<$ 當日最低價，委託單失效 (EXPIRED)。
- 資金交割：成交後立即從交割戶扣款，不足則判定破產。

(4) 盤中限價賣單

- 下單時機：玩家於盤中階段送出限價賣單。
- 撮合結算時機：當玩家點擊推進至盤後階段時執行。
- 成交條件：限價 $\leq$ 當日最高價 (high_price)。
- 實際成交價：玩家指定的限制價格。
- 未成交條件：限價 $>$ 當日最高價，委託單失效 (EXPIRED)。
- 資金交割：成交後將淨額即時入帳至交割戶。

(5) 盤中市價買單

- 下單時機：玩家於盤中階段送入市價買單。
- 撮合結算時機：當玩家點擊推進至盤後階段時執行。
- 成交條件：當日市場存在有效之成交價格資料。
- 實際成交價：當日收盤價 (close_price)，以此模擬盤中滑價的最差情況。

- **未成交條件：**若當日市場因颱風天、停牌等原因無法交易，委託單自動失效 (EXPIRED)。
- **資金交割：**成交後立即自交割戶扣款，不足則判定破產。

(6) 盤中市價賣單

- **下單時機：**玩家於盤中階段送入市價賣單。
- **撮合結算時機：**當玩家點擊推進至盤後階段時執行。
- **成交條件：**當日市場存在有效之成交價格資料。
- **實際成交價：**當日收盤價 (close_price)。
- **未成交條件：**當日無法交易時自動失效 (EXPIRED)。
- **資金交割：**成交後將淨額即時入帳至交割戶。

(7) 盤後定價買單

- **下單時機：**玩家於盤後階段 (14:00 - 14:30) 送出定價買單。
- **撮合結算時機：**當玩家點擊推進至收市階段時執行。
- **成交條件：**當日市場存在有效之收盤價格資料。
- **實際成交價：**當日收盤價 (close_price)。
- **未成交條件：**當日無法交易時自動失效 (EXPIRED)。
- **資金交割：**成交後自交割戶扣款，不足則判定破產。

(8) 盤後定價賣單

- **下單時機：**玩家於盤後階段送出定價賣單。
- **撮合結算時機：**當玩家點擊推進至收市階段時執行。
- **成交條件：**當日市場存在有效之收盤價格資料。
- **實際成交價：**當日收盤價 (close_price)。
- **未成交條件：**當日無法交易時自動失效 (EXPIRED)。
- **資金交割：**成交後將淨額即時入帳至交割戶。